Deep Learning Practical 6

Diffusion probabilistic modeling

1 Overview

Welcome to the sixth deep learning practical. In this practical, we will be implementing a minimal denoising diffusion-based probabilistic model from scratch. The structure for this practical is as follows:

- The forward process.
- The reverse process.
- U-Net with lazy layers.
- Conditional diffusion.

We will begin with MNIST, compare CIFAR-10, and extend the noise predictor with skip connections and conditioning before the image-generation challenge.

2 Load the images

Both datasets use three channels at 32×32, so switching datasets leaves the architecture unchanged. For MNIST, resize each digit and repeat its greyscale values across the three channels:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import matplotlib.pyplot as plt
import einops

from torchvision import transforms
from einops import rearrange

device = torch.device('cuda') if torch.cuda.is_available() else torch.device('cpu')

# helper functions
def cycle(iterable):
    while True:
        for x in iterable:
            yield x

def normalise(x):
    return (x-x.min())/(x.max()-x.min())

# MNIST dataloader (we set MNIST to Bx3x32x32 so our code works the same later for CIFAR10)
train_loader = torch.utils.data.DataLoader(torchvision.datasets.MNIST("./data", train=True,
    download=True,transform=transforms.Compose([
        transforms.ToTensor(),
        transforms.Resize((32,32)), transforms.Lambda(lambda x: x.repeat(3,1,1)),
        transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])), batch_size=16, shuffle=True)

train_iterator = iter(cycle(train_loader))
```

3 Forward diffusion

As described in section two of the 'Denoising Diffusion Probabilistic Models' paper (Jonathan Ho et al.), the *forward process* also called the *diffusion process* is fixed to a Markov chain that gradually adds noise to the data according to a variance $\beta_1, ..., \beta_T$ schedule of T steps. The network $\boldsymbol{\epsilon}_\theta$ is trained to estimate the noise $\boldsymbol{\epsilon}$ that data $\mathbf{x}$ is corrupted with, from random points on the variance schedule. This is the full algorithm:

Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Exercise

In line 2, sample a batch of data from the data distribution $\mathbf{x} \sim q(\mathbf{x})$. Also, for now, sample labels $\mathbf{c}$ from the dataset $\mathbf{x}, \mathbf{c} \sim q(\mathbf{x}, \mathbf{c})$ as later we will use these in an extension.

```
x,c = next(train_iterator)
x,c = x.to(device), c.to(device)
```

In line 3, specify where we are in time on the variance schedule uniformly randomly (do this for each item in the batch and send the results to the GPU device). Use $T = 400$.

```
T = 400
_ts = torch.randint(1, T+1, (x.shape[0],)).to(device)
```

In line 4, sample a tensor of noise $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. This noise tensor should be of the same rank and size as $\mathbf{x}$.

```
eps = torch.randn_like(x)
```

In line 5, you will need $\sqrt{\bar{\alpha}_t}$ and $\sqrt{1-\bar{\alpha}_t}$, where $\bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s)$. Precompute these coefficients below, then plot `sqrtab.flatten().cpu()` and `sqrtmab.flatten().cpu()` to see how the signal and noise contributions change with time:

```
beta1 = 1e-4
beta2 = 0.02
T = 400

beta_t = (beta2 - beta1) * torch.arange(0, T + 1, dtype=torch.float32, device=device) / T + beta1
beta_t[0] = 0.0  # index zero is the clean-data endpoint
alpha_t = 1 - beta_t
log_alpha_t = torch.log(alpha_t)
alphabar_t = torch.cumsum(log_alpha_t, dim=0).exp()

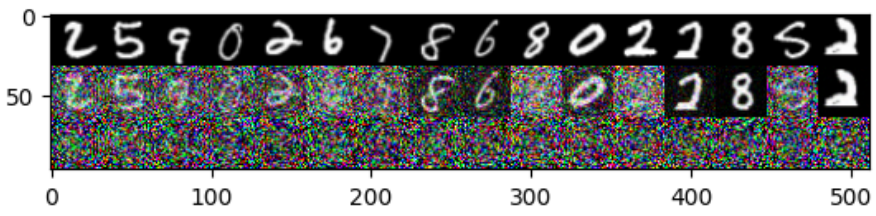
sqrtab = torch.sqrt(alphabar_t).view(-1,1,1,1)
sqrtmab = torch.sqrt(1 - alphabar_t).view(-1,1,1,1)
```

In line 5, calculate the inputs for the neural network and show a grid of images with three rows. On the first row show the original data $\mathbf{x}$. On the second row, show the batch of partially diffused data (input to the network). On the last row, show the noise $\boldsymbol{\epsilon}$ that the network tries to predict.

```
# network inputs
x_t = (sqrtab[_ts] * x + sqrtmab[_ts] * eps)
_ts/T

# data is in range -1 to +1 so we have to correct this
img_grid = rearrange(torch.cat((x*0.5+0.5, x_t*0.5+0.5, eps), dim=0),
    '(b1 b2) c h w -> (b1 h) (b2 w) c', b1=3)

plt.imshow((img_grid.cpu()*255).int().numpy())
```



Now create a DDPM class that extends `nn.Module`. Give it a `def forward(self, x)` function that returns the loss `F.mse_loss(eps, self.net(x_t))` (we are not going to also inject t into the network just yet). Initialise it with a very simple neural network `self.net` for now, e.g. `nn.Sequential` with a simple `3,1,1` pattern 2D

convolution with 64 output channels, batch normalisation, a `nn.ReLU` layer, then a final `3,1,1` convolution with 3 output channels. This means that the neural network `self.net` must output a tensor of the same rank and dimension as the input data.

```
class DDPM(nn.Module):
    def __init__(self,):
        super(DDPM, self).__init__()
        self.net = nn.Sequential(
            nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3,stride=1,padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.Conv2d(in_channels=64, out_channels=3, kernel_size=3,stride=1,padding=1)
        )

    # algorithm 1 in DDPM paper
    def forward(self, x):
        _ts = torch.randint(1, T+1, (x.shape[0],)).to(x.device)
        eps = torch.randn_like(x)
        x_t = (sqrtab[_ts] * x + sqrtmab[_ts] * eps)
        return F.mse_loss(eps, self.net(x_t))
```

Lastly, finish writing Algorithm 1 by implementing the outer training loop and optimising the neural network. Use Adam with `lr=1e-4`. Train the algorithm for 2,000 batches of data and print the loss values (better with some smoothing).

```
# train the model
ddpm = DDPM().to(device)
optim = torch.optim.Adam(ddpm.parameters(), lr=1e-4)
step, loss_smooth = 0, 0.0
ddpm.train()

while(step < 2000):

    x,c = next(train_iterator)
    x,c = x.to(device), c.to(device)

    loss = ddpm(x)
    optim.zero_grad()
    loss.backward()
    optim.step()

    loss_smooth = loss_smooth + 0.01 * (loss.item() - loss_smooth)
    if step % 100 == 0:
        print('step: {:4d} train loss: {:.3f}'.format(step, loss_smooth))

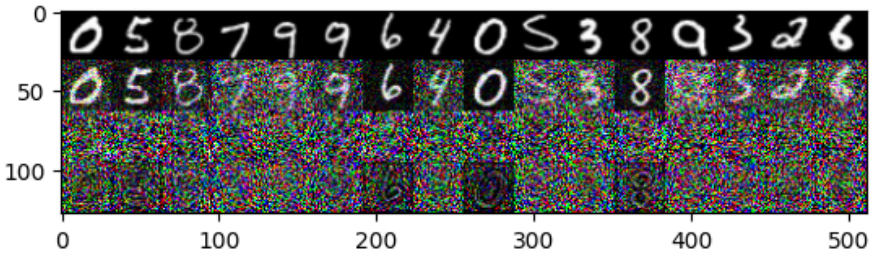
    step += 1
```

Now display a grid of images as before, but this time add a fourth row displaying the output for your model after training `ddpm.net(x_t)`. What do you expect to see? You may want to compare the outputs before—and after—training accordingly.

```
_ts = torch.randint(1, T+1, (x.shape[0],)).to(x.device)
eps = torch.randn_like(x)
x_t = (sqrtab[_ts] * x + sqrtmab[_ts] * eps)

img_grid = rearrange(torch.cat((x*0.5+0.5, x_t*0.5+0.5, eps, ddpm.net(x_t)), dim=0),
    '(b1 b2) c h w -> (b1 h) (b2 w) c', b1=4)

plt.imshow((img_grid.cpu()*255).int().numpy())
```



The network learns to estimate the noise. With a network this small, and after such a small amount of training, it won't always be perfect.

4 Reverse diffusion

We have now trained a network to estimate the added noise. The forward corruption process itself is fixed. After training, we can reverse this process—starting with noise and gradually denoising—to sample the network (a generative model). This is the algorithm we will implement:

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \ldots, 1$ **do**
3: $\quad \mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\quad \mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

As before, the variance schedule terms are a little involved so they are provided and can be precomputed:

```
oneover_sqrta = 1 / torch.sqrt(alpha_t)
mab_over_sqrtmab_inv = torch.zeros_like(beta_t)
mab_over_sqrtmab_inv[1:] = (1 - alpha_t[1:]) / sqrtmab.flatten()[1:]
sigma_t = torch.sqrt(beta_t)
```

Implement the `sample(self, n_sample, size)` function within your DDPM class. To allow more time for experimentation, here is the implementation:

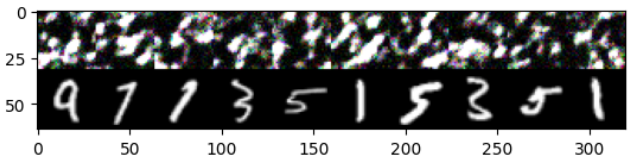
```
def sample(self, n_sample, size):
    x_i = torch.randn(n_sample, *size).to(device)
    for i in range(T, 0, -1):
        z = torch.randn(n_sample, *size).to(device) if i > 1 else 0
        eps = self.net(x_i)
        x_i = (oneover_sqrta[i] * (x_i - eps * mab_over_sqrtmab_inv[i]) + sigma_t[i] * z)
    return x_i
```

After training, you can therefore sample the model with:

```
# sample the model
ddpm.eval()

with torch.no_grad():
    xh = ddpm.sample(10, (3, 32, 32))
    img_grid = rearrange(torch.cat([(xh*0.5+0.5), (x*0.5+0.5)[:10]], dim=0),
        '(b1 b2) c h w -> (b1 h) (b2 w) c', b1=2)
    plt.imshow((img_grid.cpu()*255).int().numpy())
```

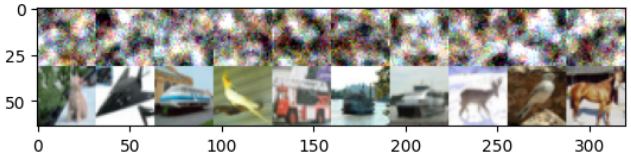
That was a lot of work. So what is the sample quality like?



The samples contain fragments of digit-like shapes, but few convincing digits. Swap the data loader to CIFAR-10, then create a fresh DDPM and optimiser and repeat the same training loop before sampling again:

```
# CIFAR10 dataloader
train_loader = torch.utils.data.DataLoader(torchvision.datasets.CIFAR10("./data",
    train=True,download=True,transform=transforms.Compose([
        transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])), batch_size=16, shuffle=True)

train_iterator = iter(cycle(train_loader))
```



The CIFAR-10 samples contain broad colour patches, but recognisable objects remain difficult to distinguish. Their superficial resemblance to small colour photographs does not establish a closer distributional match than in the MNIST experiment.

The main problem (besides only training for a very short amount of time) is that the network with two convolutional layers isn't expressive enough.

5 U-Net with lazy layers

Inside the DDPM class, replace the two-convolution network `self.net = nn.Sequential(...)` with `self.net = UNet()`, and create a new `class UNet(nn.Module)`.

Implement a U-Net with `4,2,1` downsampling and upsampling stages. Concatenate the features from each of the four encoder stages into the decoder at the matching resolution, and add a fifth connection from the input image before the final convolution. Use output channels [72, 96, 128, 196] in the encoder, followed by a 256-channel bottleneck. PyTorch's lazy layers infer their input size on the first forward pass: `nn.LazyConv2d` and `nn.LazyConvTranspose2d` infer input channels, while `nn.LazyLinear` infers input features. Try designing the network before comparing with this implementation:

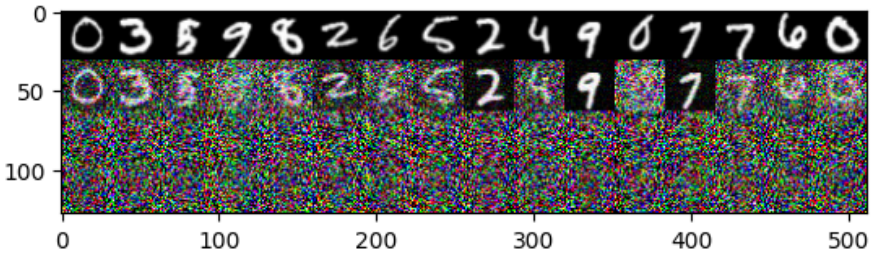
```
class UNet(nn.Module):
    def __init__(self, out_class=3):
        super().__init__()
        self.dconv_down1 = self.conv_block(72)
        self.dconv_down2 = self.conv_block(96)
        self.dconv_down3 = self.conv_block(128)
        self.dconv_down4 = self.conv_block(196)
        self.bottleneck  = nn.LazyConv2d(256, 3,1,1, bias=False)
        self.dconv_up4 = self.conv_block(256, up=True)
        self.dconv_up3 = self.conv_block(196, up=True)
        self.dconv_up2 = self.conv_block(128, up=True)
        self.dconv_up1 = self.conv_block(96, up=True)
        self.conv_last = nn.LazyConv2d(out_class, 1)

    def conv_block(self, out_channels, up=False):
        ConvType = nn.LazyConvTranspose2d if up else nn.LazyConv2d
        return nn.Sequential(
            ConvType(out_channels, 4, 2, 1, bias=False),
            nn.BatchNorm2d(out_channels),
            nn.ReLU())

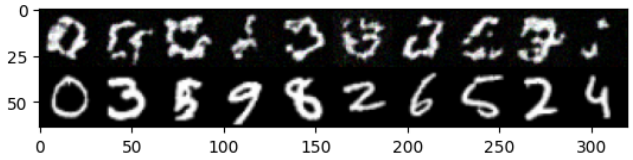
    def forward(self, x):
        start = x
        conv1 = self.dconv_down1(x)     # skip-connection 1
        conv2 = self.dconv_down2(conv1) # skip-connection 2
        conv3 = self.dconv_down3(conv2) # skip-connection 3
        conv4 = self.dconv_down4(conv3) # skip-connection 4

        x = self.bottleneck(conv4)
        x = torch.cat([x, conv4], dim=1)
        x = self.dconv_up4(x)
        x = torch.cat([x, conv3], dim=1)
        x = self.dconv_up3(x)
        x = torch.cat([x, conv2], dim=1)
        x = self.dconv_up2(x)
        x = torch.cat([x, conv1], dim=1)
        x = self.dconv_up1(x)
        x = torch.cat([x, start], dim=1)
        return self.conv_last(x)
```

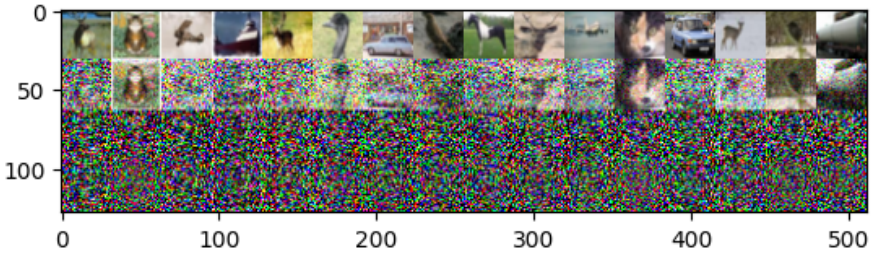
Restore the MNIST loader and iterator from Section 2, then create a fresh DDPM and optimiser using this U-Net. Train for 10,000 updates and inspect the loss and predicted noise. The example below shows closer agreement with the target noise than the smaller network, although both architecture and training duration have changed:



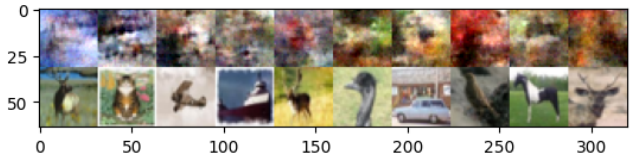
And similarly this produces better (much room for improvement) samples:



Repeat with the CIFAR-10 loader and a fresh model and optimiser. Compare the predicted noise and generated samples, treating the loss as a measure of noise-prediction error rather than a direct comparison of image quality across datasets:



And similarly this also produces better (much room for improvement) samples:



6 Conditional diffusion

The noise predictor in DDPM takes the timestep t as an input, identifying how much noise has been added. Our first examples omitted this input to keep the network small. We will now add the normalised time t/T and a class label c, allowing us to choose which digit or object class to generate.

Represent the class label by a ten-dimensional one-hot vector:

```
ch = F.one_hot(c, num_classes=10).float().to(device)
```

Concatenate the normalised time and one-hot class vector, then broadcast these 11 values across the spatial positions of two decoder stages. Recreate the U-Net before training, so its lazy layers can initialise with the extra input channels:

```
class ConditionalUNet(UNet):
    def forward(self, x, t, c):
        start = x
        conv1 = self.dconv_down1(x)     # skip-connection 1
        conv2 = self.dconv_down2(conv1) # skip-connection 2
        conv3 = self.dconv_down3(conv2) # skip-connection 3
        conv4 = self.dconv_down4(conv3) # skip-connection 4

        ch = F.one_hot(c, num_classes=10).float()
        cond = torch.cat([t[:, None], ch], dim=1)[:, :, None, None]
```

```
        x = self.bottleneck(conv4)
        x = torch.cat([x, conv4, cond.expand(-1,-1,x.size(2),x.size(3))], dim=1) # injection 1
        x = self.dconv_up4(x)
        x = torch.cat([x, conv3, cond.expand(-1,-1,x.size(2),x.size(3))], dim=1) # injection 2
        x = self.dconv_up3(x)
        x = torch.cat([x, conv2], dim=1)
        x = self.dconv_up2(x)
        x = torch.cat([x, conv1], dim=1)
        x = self.dconv_up1(x)
        x = torch.cat([x, start], dim=1)

        return self.conv_last(x)
```

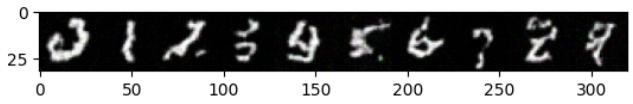
Pass time and class labels to this network in both the loss and sampler:

```
class DDPM(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = ConditionalUNet()

    def forward(self, x, c):
        t = torch.randint(1, T+1, (len(x),), device=x.device)
        eps = torch.randn_like(x)
        x_t = sqrtab[t]*x + sqrtmab[t]*eps
        return F.mse_loss(eps, self.net(x_t, t/T, c))

    @torch.no_grad()
    def sample(self, n_sample, size):
        self.eval()
        x_i = torch.randn(n_sample, *size, device=device)
        c = torch.arange(n_sample, device=device) % 10
        for i in range(T, 0, -1):
            t = torch.full((n_sample,), i/T, device=device)
            eps = self.net(x_i, t, c)
            z = torch.randn_like(x_i) if i > 1 else 0
            x_i = oneover_sqrta[i]*(x_i - eps*mab_over_sqrtmab_inv[i]) + sigma_t[i]*z
        return x_i
```

For the MNIST comparison, restore its loader and iterator, then create a fresh DDPM and optimiser. Use `loss = ddpm(x, c)` in the earlier loop. Example class-conditioned MNIST samples after 25,000 updates:



The samples show recognisable class information, with some malformed digits. Compare this concatenation method with the time and class embeddings in the conditional diffusion example. You can also investigate residual blocks, sinusoidal time embeddings and attention within the U-Net.

7 Image flow matching

To construct a straight path from noise to an image, draw a clean image x_1 and independent Gaussian noise x_0, and form

$$x_t = (1-t)x_0 + tx_1, \qquad u_t = x_1 - x_0, \qquad L(\theta) = \mathbb{E}\left[\frac{1}{3\cdot 32^2}\|v_\theta(x_t,t,c) - u_t\|_2^2\right].$$

Using a fresh `ConditionalUNet`, we can predict the velocity along this path. The loop below creates its own model and optimiser, then trains on the images and labels returned by `train_iterator`. As in the equation, `F.mse_loss` averages over all image values:

```
net = ConditionalUNet().to(device)
optim = torch.optim.Adam(net.parameters(), lr=1e-4)
net.train()
for step in range(2000):
    x, c = next(train_iterator)
    x, c = x.to(device), c.to(device)
    x0 = torch.randn_like(x)
    t = torch.rand(len(x), device=x.device)
    t_view = t[:, None, None, None]
    xt = (1-t_view)*x0 + t_view*x
    velocity_target = x-x0
    loss = F.mse_loss(net(xt, t, c), velocity_target)
    optim.zero_grad()
    loss.backward()
    optim.step()
```

During generation the clean target is unavailable. Initialise x with Gaussian noise and integrate the learned field:

```
net.eval()
with torch.no_grad():
    x = torch.randn(16, 3, 32, 32, device=device)
    c = torch.arange(16, device=device) % 10
    for k in range(50):
        t = torch.full((16,), k/50, device=device)
        x = x + net(x, t, c)/50
```

Compare samples with DDPM at the same number of training updates. Draw a one-pixel straight path by hand; what is its derivative? Why can we use $x_1 - x_0$ in training but not simply use that expression in the sampler? Compare 10, 25 and 50 integration steps.

8 Constrained generative modelling

The submission uses the same CIFAR-10 horse subset as Practical 5: 4,000 training photographs and 1,000 validation photographs, at $3 \times 32 \times 32$. Use DDPM or image flow matching to generate new horses. Compare the colours, viewpoints, poses, legs and backgrounds with the GAN examples. The MNIST and full CIFAR-10 experiments above provide the implementation steps for this challenge.

In `dl-practical6.py`, choose `task="ddpm"` or `task="flow"` in `CONFIG`. The 120,075-parameter U-Net starts with 4,000 updates and takes time and a class label; for this single-class task, the label is always zero. Improve it using the residual blocks, skip connections and conditioning explored above. The submission fixes sampling at 100 DDPM steps or 25 Euler steps.

Both challenges allow 1,000,000 parameters and buffer elements, and 20,000 optimiser updates in total. For the GAN, this means 10,000 iterations with one update of each network. Compute and real-image exposure still differ, so compare training and sampling times alongside image quality.

Score the generated horses with the same KID procedure as Practical 5. The scores below report KID multiplied by 1,000; lower scores indicate a closer match. Compare sample diversity and structure alongside the score.

Exercises

1. Run the short DDPM and flow baselines. Compare their outputs with the real horses and with your GAN. Which structures appear first, and what differences remain?

2. Add another spatial scale to the U-Net. Compare allocating channels at full resolution with allocating them near the bottleneck, using a similar parameter count.

3. Examine the way the supplied residual block injects time. Compare adding a time embedding with predicting a scale and shift for normalised features.

4. Compare an architectural change at the same number of training updates before increasing training length. Keep the full loss trace and sample grids while investigating the result.

Two baseline methods

Figures 1 and 2 show two alternative ways to generate horses, trained separately on the same data with the same U-Net architecture, 4,000 updates and seed 3801. DDPM learns to predict noise; flow matching learns the velocity from noise to an image. Both grids display the generated RGB pixels directly, so their different colour casts come from the models. Compare sample quality and KID; the training losses have different targets, so their values are not directly comparable.

DDPM baseline: 120,075 parameters, 4,000 updates, batch 64; KID x 1000: 220.038.



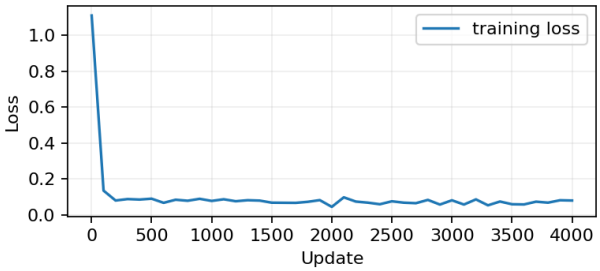


Figure 1: DDPM baseline (100 denoising steps): generated horses (left) and training loss (right).

Flow-matching baseline: 120,075 parameters, 4,000 updates, batch 64; KID x 1000: 202.923.



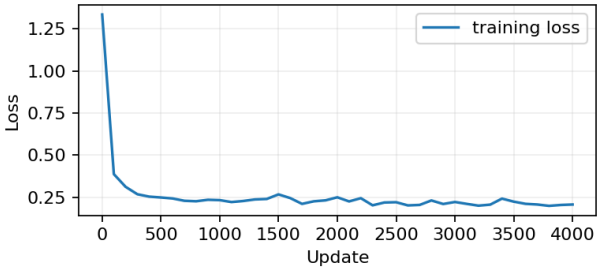


Figure 2: Flow-matching baseline (25 Euler steps): generated horses (left) and training loss (right).

What changes can achieve

Here is an example produced within the assignment constraints, with KID $\times 1000 = 30.596$.



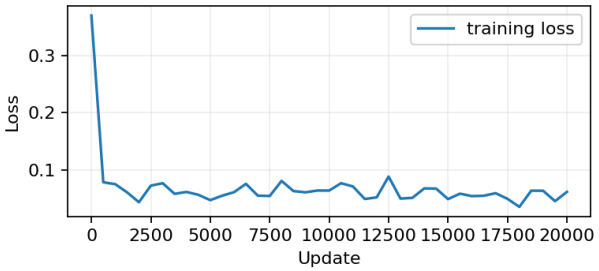


Figure 3: Example: generated horses (left) and training loss (right).

9 Submission

Download `dl-practical6.py`, set `STUDENT_ID` and edit the model and training settings. Run the complete file or paste the whole program into one notebook cell. It trains, scores and creates the ZIP below; upload this to the corresponding Ultra item.

```
lab-06-studentID.zip
  lab-06-code.py
  lab-06-results.json
  samples/
    batch-01.png
    batch-02.png
    ...
    batch-10.png
```

Details and constraints

- Generate ten batches of 100 model samples, arranged as 10 by 10 grids. Each PNG is 320 by 320 RGB 8-bit pixels, with no padding or resizing. These 1,000 images are also used for KID.
- Use at most 1,000,000 parameters and buffer elements in total, 20,000 optimiser updates and a batch size of at most 64.
- Train from fresh weights on the fixed 4,000 CIFAR-10 horse training images at 32 by 32 RGB. Reserve the Inception network for evaluation.
- Use no extra data, pretrained models, distillation teacher, gradient accumulation or extra training batches per update.
- Choose DDPM or image flow matching. Evaluate against the 1,000 validation images; JSON stores raw KID and the printed score is KID x 1,000.
- Put a brief abstract at the top of the Python file, with references and AI acknowledgements in comments. The program writes the JSON and sample files; submit these unchanged.

Marking

- Marks reflect the quality and simplicity of your solution and its measured performance within the constraints. Aim for a KID x 1,000 below the corresponding baseline: 220.0 for DDPM or 202.9 for image flow matching.
- A pass requires a small improvement over the baseline. A 2:1 requires significant improvements using established techniques discussed in the course. A high first requires substantial improvements through well-implemented state-of-the-art methods, for example by integrating methods from recent papers, producing high-quality samples and scores within the budget.
- The exact marking function is withheld to avoid gamification.

Solutions for this assignment cannot later be publicised without request. Requests will be considered on a case-by-case basis after marking.